Lehrstuhl für Software und Systems Engineering
TUM School of Computation, Information and Technology
Technical University of Munich

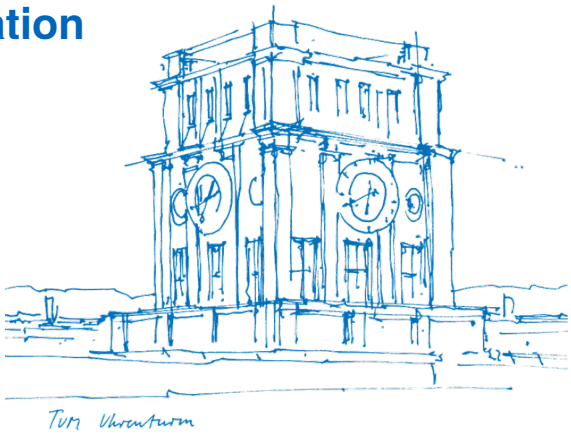
Agentic AI for Code Generation

Seminar Report — SS26

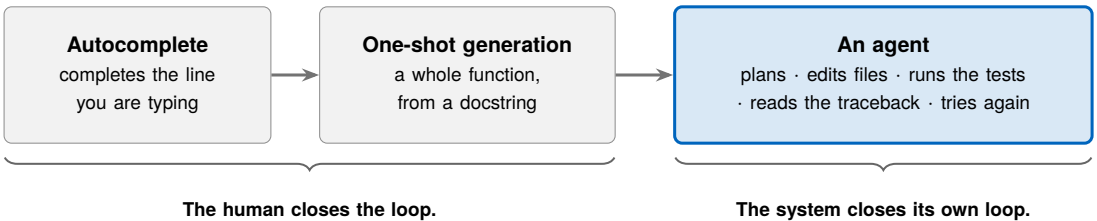
Thua Duc Nguyen

Supervisor: Derui Zhu
TUM School of Computation, Information and Technology, Technical University of Munich

July 19, 2026

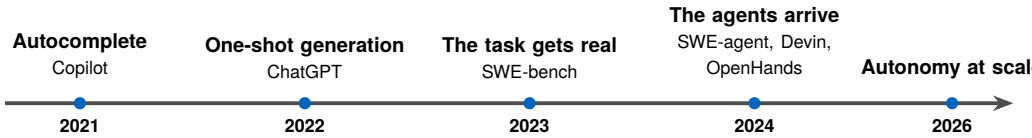


What is agentic AI for code generation?



Plain language in — a code change out.

Three years, three eras

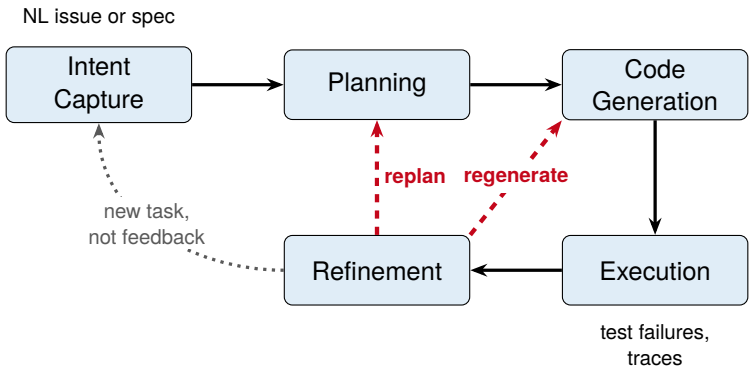


under 2 % ⟶ **over 70 %**

of real GitHub issues resolved autonomously [10, 21]

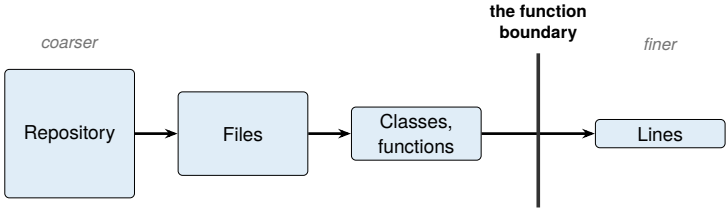
(two different rulers — defined in a few slides)

The agent pipeline



Five stages — **one vocabulary for every system in this talk.**

Planning is localisation, not design

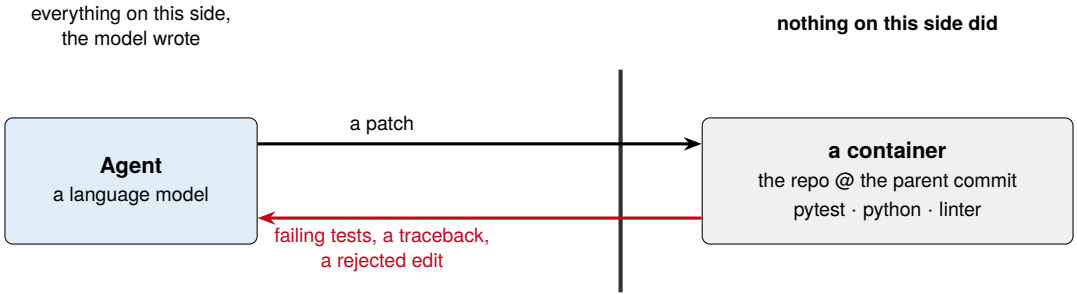


Above it, localisation *is* the task — ranking functions *helps* [26].

Below it, it hurts — *no* line pointer beats *perfect* localisation [24].

Reasoning about *where* the code is — not *how* to write it.

Execution: the code meets the environment



Not a score — *a symptom*, still to be read.

Refinement: what to do with the signal

- **Repair from the trace** [9]
- **Explain your own code** [3] — *only where tests exist*
- **Sample N, rerank with a trained critic** [16]

Agents improve their own code.

What SWE-bench measures

One instance [10] = a real merged PR: closed an issue + touched the tests.

The agent sees the repo before the merge + the issue; **returns** a patch.

Graded by running the tests — the bug fixed, nothing broken.

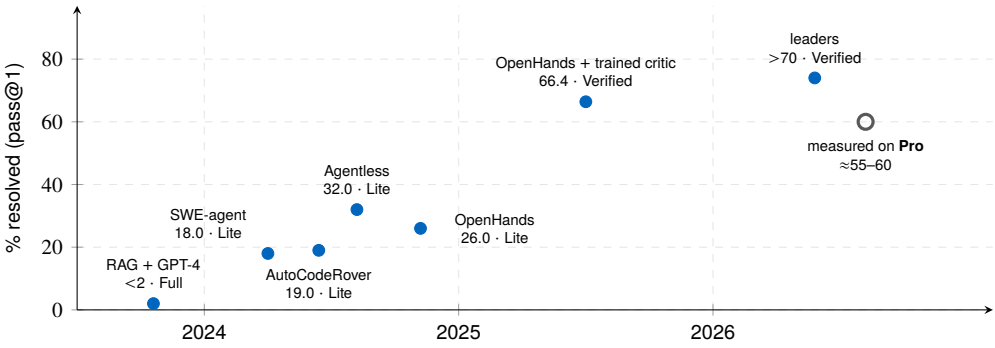
Resolve rate = both, on the first patch.
Every number from here on is this one.

One benchmark, four splits:

- **Full** — 2,294, every scraped PR
- **Lite** — 300, single-file patches
- **Verified** — 500, human-screened
- **Pro** — 1,865, held-out repos

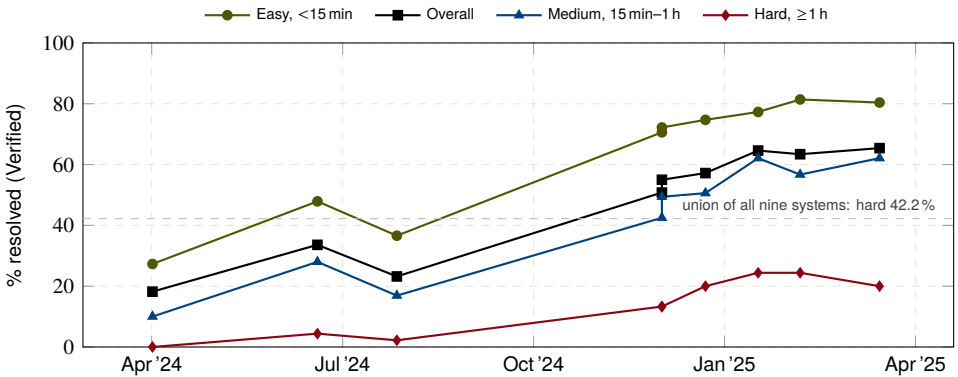
Rates are not comparable across splits. Verified is the most *curated* split, not the hardest.

Three years, in numbers



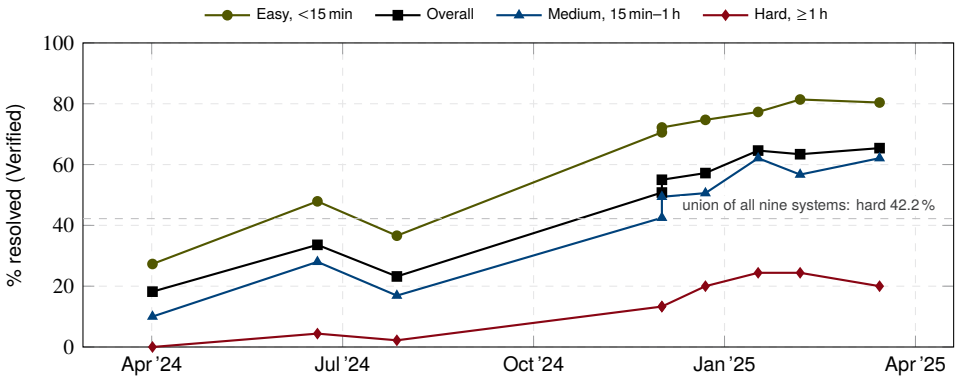
Splits are **not comparable** — read the *climb*, not the deltas. [10, 25, 26, 23, 22, 16, 21, 17]

What the climb hides



Nine systems with published per-tier results; tiers = annotator-estimated fix time ($n = 194/261/45$ of 500) [6, 14].

What the climb hides



Nine systems with published per-tier results; tiers = annotator-estimated fix time ($n = 194/261/45$ of 500) [6, 14].

A system can gain ten points — and solve no additional hard task.

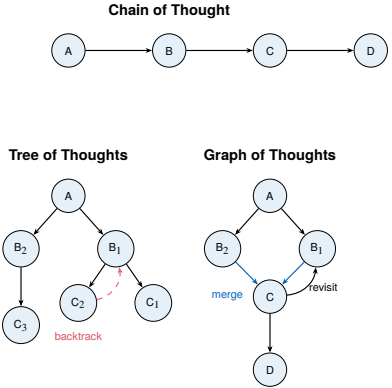
Conclusion

- In three years: under 2 % → **over 70 %**.
- **What changed is the loop** — the system runs its own code, and reads what comes back.
- The climb was mostly the easy strata — **hard tasks stalled near 20 %** [6]. *The climb is not over.*

Backup: the numbers behind the stage walk

- **Planning, above the function boundary** [26]. AutoCodeRover adds spectrum-based fault localisation to rank *functions*: $+3$ **points** on SWE-bench Lite.
- **Planning, below it** [24]. D4C hands the model the failing test and the *whole function*, and gives it *no line-level pointer at all* — beating systems given perfect localisation by **10 %**. Below the function boundary, a pointer fights the model's infilling prior.
- **Refinement, with an explanation** [3]. Self-debug: up to $+12\,\%$ — but only where a test suite exists to run against.
- **Refinement, with a trained critic** [16]. OpenHands on Verified, best-of-N reranked: 60.6 → **66.4 %**. (On the deck: the numbers timeline.)

Backup: how are plans shaped?



The evaluator, not the branching, makes the structure useful — and it is what a repository-scale agent finds hardest to supply. *What is a half-finished patch worth, before any test has run?* SWE-Search [1] builds one (MCTS + LLM value function, $+23\,\%$ relative) — and must invoke a model at every node.

Backup: how do agents refine?

Mechanism	Closed by	Signal	Characteristic failure
Self-Refine [12]	**self**	Model critiques own output	Plateaus; nothing can contradict the generator
Self-debug, no tests [3]	**self**	Self-generated explanation	Explains the bug as intended behaviour
Self-debug, with tests [3]	**oracle**+**self**	Test verdict + model diagnosis	Bounded by test coverage
Reflexion [18]	**oracle** / **proxy**	Task reward where one exists; on HumanEval, self-generated tests	Filed as external, but its code result rests on a proxy
SelfEvolve [9]	**oracle**	Interpreter errors, traces	Silent (non-crashing) semantic bugs
D4C [24]	**oracle**	Failing tests + full function	Plausible-but-incorrect patches
CRITIC [8]	**oracle**	Self-critique routed *through* an external tool	Needs a tool covering the error class
Agentless validate [23]	**oracle**+**proxy**	Regression tests + generated reproduction tests	The proxy half inherits the model's misreading
Best-of-N + critic [16]	**oracle**+**proxy**	Executed rollouts ranked by a trained critic	Critic is a distilled proxy; cost scales with N

oracle = a real checker — it runs the code. **proxy** = a made-up checker — trained or generated. **self** = the model judging itself.

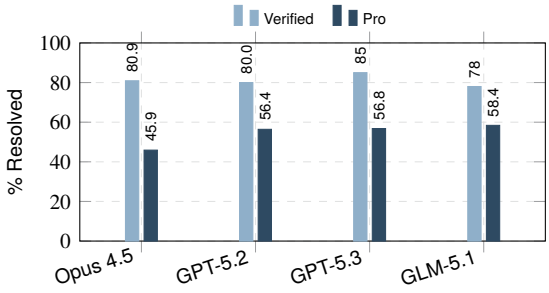
Backup: does the scaffold matter?

	SWE-agent custom ACI	**OpenHands** code-as-action	**mini-SWE-agent** ~100 lines of Python	**spread**
GPT-5-mini	38.8	38.6	**51.8**	13.2
Qwen3 Coder 480B	36.6	**54.0**	40.2	17.4
DeepSeek R1	**22.6**	18.0	11.0	11.6

Resolve rate (pass@1) on SWE-bench Verified, April 2026 [19, 20]. ACI = agent–computer interface: the command set the model is handed.

- Same model, same benchmark: the scaffold alone is worth **11–17 points**.
- **No scaffold wins twice** — the best scaffold *depends on the model*.

Backup: is the 70 % real?



The *same weights* drop ≈ 20–35 points on the contamination-resistant SWE-bench Pro [4].

Read only the floor. Pro is harder *as well as* fresher, and the column mixes harnesses — so the drop *size* is not a measurement of memorisation.

91 % of Verified is work a developer finishes *within the hour* [6]. A system can gain points while solving no additional hard task.

July 2026: the ruler itself is under audit. OpenAI flags ∼30 % of Pro's public tasks as broken and retracts its February recommendation to use it [15]. Quote *measured* Pro numbers, with that caveat attached.

Backup: what does running an agent cost?

- **Longer windows do not dissolve the bottleneck.** Retaining full *dependency* context works best; padding with loosely related code *degrades* correctness [13].
- **Context explosion and semantic drift.** Appending every observation buries the critical early information [11].
- **Cost is quadratic in turns.** The whole prefix is re-sent each turn, so tokens grow as $O(n^2)$ [7]. A property of the protocol, not an empirical finding; prompt caching cuts the constant, not the growth rate.
- **Resolve rate decouples from compute** [5]. Agentless leads on resolve rate (48 % with Qwen3-32B) at 83.1 API calls and 209.4 s per issue. The frontier is accuracy *per dollar*.
- **And verification is the human bottleneck.** In an RCT, experienced developers were 19 % *slower* with early-2025 AI tools — while believing they had been 20 % faster [2].

References I

[1] A. Antoniades, A. Örwall, K. Zhang, Y. Xie, A. Goyal, and W. Wang.
SWE-search: Enhancing software agents with monte carlo tree search and iterative refinement.
arXiv preprint arXiv:2410.20285, 2024.
ICLR 2025.

[2] J. Becker, N. Rush, E. Barnes, and D. Rein.
Measuring the impact of early-2025 AI on experienced open-source developer productivity.
arXiv preprint arXiv:2507.09089, 2025.
METR randomised controlled trial.

[3] X. Chen, M. Lin, N. Schärli, and D. Zhou.
Teaching large language models to self-debug.
In *The Twelfth International Conference on Learning Representations*, 2024.

[4] X. Deng et al.
SWE-bench pro: Can AI agents solve long-horizon software engineering tasks?
arXiv preprint arXiv:2509.16941, 2025.

[5] Z. Fan, K. Vasilevski, D. Lin, B. Chen, Y. Chen, Z. Zhong, J. M. Zhang, P. He, and A. E. Hassan.
SWE-effi: Re-evaluating software AI agent system effectiveness under resource constraints.
arXiv preprint arXiv:2509.09853, 2025.

[6] J. Ganhotra.
Cracking the code: How difficult are SWE-bench-verified tasks really?
`https://jatinganhotra.dev/blog/swe-agents/2025/04/15/swe-bench-verified-easy-medium-hard.html`, 2025.
Blog post, accessed: 2025-06-01.

[7] P. Gao and C. Peng.
More with less: An empirical study of turn-control strategies for efficient coding agents.

References II

arXiv preprint arXiv:2510.16786, 2025.

[8] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen.
CRITIC: Large language models can self-correct with tool-interactive critiquing.
In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
arXiv:2305.11738.

[9] S. Jiang, Y. Wang, and Y. Wang.
SelfEvolve: A code evolution framework via large language models.
arXiv preprint arXiv:2306.02907, 2023.

[10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan.
SWE-bench: Can language models resolve real-world GitHub issues?
arXiv preprint arXiv:2310.06770, 2024.
ICLR 2024.

[11] S. Liu, J. Yang, B. Jiang, Y. Li, J. Guo, X. Liu, and B. Dai.
Context as a tool: Context management for long-horizon SWE-agents.
arXiv preprint arXiv:2512.22087, 2025.

[12] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, et al.
Self-refine: Iterative refinement with self-feedback.
In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[13] N. L. H. Nam et al.
On the impacts of contexts on repository-level code generation.
In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.

[14] OpenAI.

References III

Introducing SWE-bench verified.
https://openai.com/index/introducing-swe-bench-verified/, 2024.
Accessed: 2025-06-01.

[15] OpenAI.
Separating signal from noise in coding evaluations.
https://openai.com/index/separating-signal-from-noise-coding-evaluations/, 2026.
8 July 2026. Accessed: 2026-07-14.

[16] OpenHands.
SOTA on SWE-Bench verified with inference-time scaling and critic model.
https://www.openhands.dev/blog/sota-on-swe-bench-verified-with-inference-time-scaling-and-critic-model, 2025.
Accessed: 2026-05-28.

[17] Scale AI.
SWE-bench Pro public leaderboard.
https://labs.scale.com/leaderboard/swe_bench_pro_public, 2026.
Top measured entries, July 2026: Muse Spark 1.1 at 61.5%, GPT-5.4 at 59.1% (mini-swe-agent harness). Accessed: 2026-07-19.

[18] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao.
Reflexion: Language agents with verbal reinforcement learning.
In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, 2023.

[19] M. Suri, X. Li, M. Shojaie, S. Han, C.-C. Hsu, S. Garg, A. Deshmukh, and V. Kumar.
CodeScout: Contextual problem statement enhancement for software agents.
2026.
arXiv:2603.05744v2, 9 April 2026. Scaffold×model grid is the paper's no-augmentation baseline (its Figure 3), not its contribution.

References IV

[20] SWE-agent Team.
mini-SWE-agent.
https://github.com/SWE-agent/mini-swe-agent, 2026.
~100 lines of Python for the agent class; reports >74% on SWE-bench Verified. Accessed: 2026-07-12.

[21] SWE-bench Team.
SWE-bench leaderboards.
https://www.swebench.com. Underlying evaluation artefacts at https://github.com/SWE-bench/experiments, 2026.
Accessed: 2026-07-09.

[22] X. Wang, B. Li, Y. Song, F. F. Xu, G. Neubig, et al.
OpenHands: An open platform for AI software developers as generalist agents.
In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
arXiv:2407.16741.

[23] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang.
AGENTLESS: Demystifying LLM-based software engineering agents.
arXiv preprint arXiv:2407.01489, 2024.

[24] J. Xu, Y. Fu, S. H. Tan, and P. He.
Aligning the objective of LLM-based program repair.
In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025.
D4C. arXiv:2404.08877.

[25] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press.
SWE-agent: Agent-computer interfaces enable automated software engineering.
arXiv preprint arXiv:2405.15793, 2024.

References V

[26] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury.
AutoCodeRover: Autonomous program improvement.
In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.